DOCUMENTACIÓN DE PROPUESTA TÉCNICA Y ECONÓMICA

LAVA RÁPIDO VEHICULAR

Plataforma para la gestión de reservas, pagos, operadores y servicios de autolavado

INTEGRANTES

José Stiven Perdomo Martínez

Santiago Gordo Pérez

SERVICIO NACIONAL DE APRENDIZAJE – SENA

Tecnólogo en Análisis y Desarrollo de Software

Ficha: 3145555

2026

Contenido

1. Resumen ejecutivo

2. Introducción y justificación

3. Objetivos y alcance

4. Usuarios y escenarios de uso

5. Requisitos y criterios de aceptación

6. Arquitectura y decisión arquitectónica

7. Tecnologías seleccionadas y justificación

8. Componentes e integración

9. Modelo de datos y consistencia

10. Seguridad, privacidad y accesibilidad

11. Infraestructura y ambientes

12. Plan de trabajo y entregables

13. Equipo y responsabilidades

14. Estimación económica histórica

15. Pruebas y criterios de validación

16. Mantenimiento, riesgos y sostenibilidad

17. Conclusiones

18. Referencias

Nota editorial. Esta propuesta describe las tecnologías y componentes identificados en los repositorios al 4 de noviembre de 2026. Cuando no existe comprobación de ejecución o de un entorno desplegado, se señala expresamente la limitación.

1. Resumen ejecutivo

Lava Rápido Vehicular es una plataforma para organizar la operación de un autolavado mediante una aplicación móvil para clientes y operadores, un panel administrativo, una API REST y una base de datos relacional. Permite consultar servicios, gestionar vehículos, crear reservas, efectuar pagos en línea, asignar trabajos, seguir estados operativos y registrar calificaciones. La operación se apoya en reglas de autorización, horario y consistencia de datos.

La solución implementada responde a una arquitectura cliente-servidor con backend monolítico modular. Las tecnologías principales identificadas son Expo SDK 58, React Native 0.88 y TypeScript para la aplicación móvil; React 19, Vite 6 y TypeScript para el panel; Java 17, Spring Boot 4.0.9 y Spring Security para la API; PostgreSQL 16 y Liquibase para persistencia; Wompi para pagos; y Docker para ambientes reproducibles.

Esta propuesta justifica las decisiones técnicas, describe la integración y delimita su verificación. El código de los principales flujos se encuentra identificado, pero la consulta de una base desplegada y las transacciones reales con Wompi, SMTP o Expo Push no forman parte de la evidencia técnica disponible para esta versión.

2. Introducción y justificación

En un autolavado, los registros manuales pueden provocar cruces de horarios, inconsistencias de precios, dificultades para asignar personal y poca visibilidad de los pagos. El proyecto propone un sistema centralizado en el que el backend conserva las reglas del negocio y la persistencia, mientras los clientes móvil y web presentan las funciones correspondientes a cada rol.

La elección de tecnologías consolidadas, una base relacional y migraciones versionadas reduce la complejidad de mantenimiento del proyecto formativo. Se priorizan integridad, separación de responsabilidades, seguridad por roles y trazabilidad de transacciones frente a una distribución innecesaria en múltiples servicios.

3. Objetivos y alcance

3.1 Objetivo general

Describir y justificar la solución técnica de Lava Rápido Vehicular para soportar la gestión de reservas, pagos, operadores y servicios de lavado mediante clientes móvil y web, una API protegida y persistencia relacional.

3.2 Objetivos específicos

- Precisar las tecnologías usadas y las razones de selección.
- Describir la arquitectura, sus módulos, contratos e integraciones.
- Definir controles técnicos de seguridad, integridad y disponibilidad.
- Vincular requisitos del SRS con componentes y criterios de aceptación.
- Diferenciar la implementación observada de las validaciones operativas pendientes.

3.3 Alcance

Incluye autenticación, catálogo, vehículos, reservas, cancelaciones, pagos Wompi, operadores, asignaciones, estados operativos, comprobantes, calificaciones, preferencias y notificaciones. Quedan fuera el rastreo GPS de operadores, la optimización de rutas, inventario físico y contabilidad detallada. Los mapas existentes no representan seguimiento GPS en tiempo real.

4. Usuarios y escenarios de uso

Tabla 1. Perfiles y principales procesos

Rol	Canal	Operaciones principales
Cliente	Aplicación móvil	Registro, catálogo, vehículos, reserva, cancelación, pago, seguimiento y calificación
Operador	Aplicación móvil	Consultar sus asignaciones, iniciar y finalizar atención
Administrador	Panel web	Administrar catálogo, vehículos, marcas, operadores, asignaciones y pagos

La aplicación y el panel consumen la misma API. La autorización de las operaciones protegidas se ejecuta en el backend; las restricciones visuales no reemplazan sus comprobaciones de permisos.

5. Requisitos y criterios de aceptación

El SRS del proyecto define requisitos funcionales RF1–RF13 y requisitos no funcionales RNF1–RNF12. La propuesta los agrupa sin sustituir la especificación detallada.

Tabla 2. Trazabilidad de capacidades técnicas

Requisitos	Capacidad	Criterio técnico de aceptación
RF1	Identidad y acceso	Autenticación, permisos por rol y titularidad; recuperación segura
RF2–RF4	Catálogo	Consulta y mantenimiento de servicios con validación de datos
RF5	Calificaciones	Solo cliente titular de reserva finalizada; una por reserva
RF6–RF7	Pagos y comprobante	Confirmación verificable con proveedor; comprobante únicamente si se aprueba
RF8, RF11	Reservas	Horario 06:00–20:00, no solapamiento por vehículo y estados válidos
RF9–RF10	Perfil, vehículos y preferencias	Operaciones autorizadas y persistencia de información
RF12	Operadores	Asignación autorizada y actualización coherente de estados
RF13	Notificaciones	Historial consultable y entrega condicionada a proveedor configurado

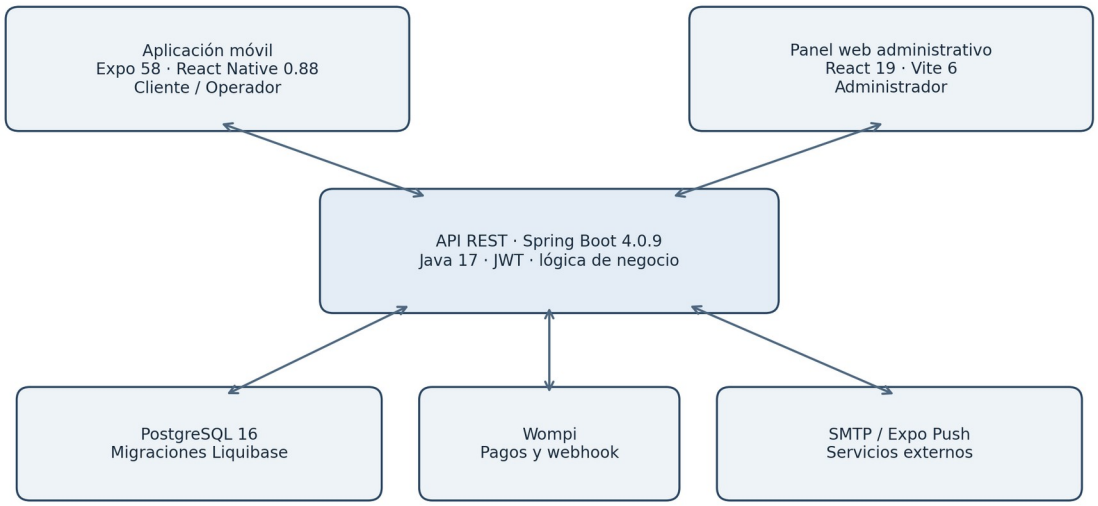
Los requisitos de calidad incluyen compatibilidad, usabilidad, seguridad, disponibilidad objetivo, rendimiento, escalabilidad, mantenibilidad y auditoría. Una meta de disponibilidad no debe confundirse con un resultado medido.

6. Arquitectura y decisión arquitectónica

6.1 Arquitectura general

El software utiliza arquitectura cliente-servidor, con dos interfaces independientes conectadas por HTTP(S) a una API REST. El backend se organiza en un monolito modular: dominios funcionales separados por paquetes que comparten el proceso de Spring Boot y una base PostgreSQL. No se identificó una arquitectura de microservicios.

Figura 1. Arquitectura lógica de Lava Rápido Vehicular



Nota. Representación conceptual elaborada a partir de los componentes y comunicaciones descritos en el proyecto; no representa una topología de infraestructura certificada.

6.2 Justificación del monolito modular

Esta decisión facilita transacciones consistentes entre reservas, pagos y asignaciones; simplifica despliegue, pruebas y observabilidad para el tamaño actual del equipo. La división por dominios permite evolucionar el código sin asumir los costos de comunicación, orquestación y consistencia distribuida de microservicios. Es una decisión adecuada al alcance actual, no una limitación permanente del producto.

6.3 Responsabilidades por capa

Los controladores definen contratos HTTP; los servicios aplican reglas del negocio; los repositorios realizan acceso a datos; las entidades expresan persistencia JPA; los DTO delimitan entradas y respuestas. Spring Security centraliza filtros de autenticación y autorizaciones. La UI no accede directamente a la base de datos.

7. Tecnologías seleccionadas y justificación

La selección se presenta conforme a los archivos de configuración revisados. Las versiones descritas corresponden al estado de los repositorios, no necesariamente a los entornos publicados.

Tabla 3. Matriz de selección tecnológica y criterio de decisión

Componente	Tecnología y versión declarada	Justificación técnica

Móvil	React Native 0.88; Expo SDK 58; TypeScript	Una base de componentes nativos con herramientas Expo para desarrollo, compilación y gestión de aplicaciones Android; tipos estáticos en contratos.
Web	React 19; Vite 6; TypeScript	Componentes reutilizables, separación por funcionalidades y ciclo de desarrollo rápido para el panel administrativo.
API	Java 17; Spring Boot 4.0.9	Ecosistema para API REST, validación, transacciones y servicios de dominio con responsabilidades claras.
Acceso	Spring Security; JWT; BCrypt	Autenticación sin sesión de servidor y verificación de autorización; almacenamiento de hashes de contraseñas.
Persistencia	PostgreSQL 16; JPA/Hibernate	Integridad referencial, índices y restricciones que protegen estados y relaciones del negocio.
Evolución BD	Liquibase	Cambios de esquema versionados, con orden de aplicación, trazabilidad y validación de migraciones.
Pagos	Wompi	Proveedor de pagos en línea con verificación de transacciones y recepción de eventos autenticados.
Infraestructura	Docker / Compose; Render documentado	Ambientes reproducibles y posibilidad de publicar servicios; el despliegue activo requiere verificación independiente.
Comunicación	Axios; REST/JSON; HTTPS	Contratos simples entre clientes y servidor, con errores HTTP explícitos y transporte cifrado.
Notificaciones	Expo Push / SMTP, según configuración	Canales independientes para notificación móvil y recuperación de contraseña; la entrega real depende del ambiente.

7.1 Comparación de alternativas relevantes

PostgreSQL se prioriza frente a una base no relacional por el dominio fuertemente conectado (usuario–vehículo–reserva–pago–asignación) y la necesidad de restricciones de integridad. Spring Boot con Java se utiliza frente a un backend JavaScript considerado inicialmente debido a la implementación ya consolidada de transacciones, seguridad y repositorios. React Native con Expo permite compartir lógica entre pantallas móviles sin mantener dos aplicaciones nativas independientes. Estas comparaciones justifican la solución concreta, no afirman superioridad absoluta de una tecnología.

7.2 Mantenibilidad y riesgos de dependencias

Las dependencias requieren actualizaciones controladas y validación en entornos de prueba. En particular, la documentación del proyecto debe sincronizarse con el manifiesto móvil, que declara Expo SDK 58 y React Native 0.88, y con Spring Boot 4.0.9 en el backend.

8. Componentes e integración

Tabla 4. Módulos principales y responsabilidad

Módulo	Responsabilidad / comunicación
auth / users	Registro, login, perfil, recuperación y políticas de acceso
servicios / marcas / vehiculos	Catálogo, gestión de vehículos y titularidad
reservas	Disponibilidad, solapamiento, cancelación y estados
operadores / asignaciones	Personal activo, asignación única y ejecución
pagos / pago_intentos	Intentos, Wompi, webhook, confirmación y conciliación
calificaciones	Opiniones asociadas a reservas finalizadas
notificaciones	Historial, lectura y entrega a proveedores configurados
auditoria / log_errores	Entidades y servicios de trazabilidad con cobertura limitada identificada

8.1 Flujo de reserva

La aplicación envía vehículo, servicio, fecha y hora. El backend valida usuario y propiedad, actividad de los registros, horario de 06:00–20:00 y solapamiento; conserva precio y duración pactados y crea una reserva PENDIENTE. Las reglas se verifican en servicio y el esquema agrega restricciones para evitar intervalos incompatibles por vehículo.

8.2 Flujo de pago

Para una reserva apta, el backend genera o reutiliza un intento y construye la referencia, monto y firma de integridad para el widget Wompi. El resultado presentado al cliente no acredita por sí solo aprobación: el backend verifica identificador, referencia, moneda COP, ambiente y valor mediante consulta remota o evento válido. La confirmación del pago permanece separada del estado operativo de la reserva.

8.3 Flujo de asignación y notificaciones

Un administrador selecciona reserva y operador activo; el backend valida condiciones y registra la asignación. El operador consulta únicamente sus tareas y actualiza el inicio o cierre de atención. Los

eventos pertinentes generan notificaciones persistidas; la entrega mediante correo o push depende de la configuración y la disponibilidad de proveedores.

9. Modelo de datos y consistencia

El repositorio de persistencia declara PostgreSQL 16, un conjunto de 24 changeSets en Liquibase y 18 tablas en las migraciones activas, entre ellas users, roles, vehiculos, servicios, reservas, pagos, pago_intentos, operadores, asignaciones, calificaciones y notificaciones. El conteo describe los archivos declarados; no prueba el historial de cambios de una instancia en producción.

Los datos se relacionan mediante claves primarias y foráneas; se definen restricciones UNIQUE y CHECK, índices y una restricción de exclusión GiST para impedir solapamientos activos del mismo vehículo. La unicidad de referencias de intento y de ciertas transacciones protege la trazabilidad de pagos. JPA utiliza validación de esquema mediante ddl-auto=validate; las migraciones son la fuente de evolución del esquema.

La aplicación efectiva de changeSets, los registros de DATABASECHANGELOG, las copias de seguridad y el contenido de una base desplegada requieren inspección autorizada del entorno real.

10. Seguridad, privacidad y accesibilidad

10.1 Autenticación y autorización

El backend utiliza Spring Security con JWT y validación de permisos de rol y titularidad. Las contraseñas se protegen con BCrypt y la recuperación usa tokens temporales de un solo uso, almacenados de forma no reversible. Existe limitación de intentos en rutas de autenticación; el mecanismo en memoria y por instancia debe reevaluarse si se escala horizontalmente.

10.2 Protección de integraciones

El transporte externo debe ser HTTPS; Wompi requiere verificación de datos y firma de eventos antes de aplicar estados. Las credenciales privadas se mantienen fuera de aplicaciones cliente, documentación y repositorios. El panel utiliza sessionStorage para su sesión, lo que exige mantener controles estrictos frente a ejecución de scripts no autorizados.

10.3 Privacidad, usabilidad y accesibilidad

La plataforma debe mostrar errores de validación, ausencia de datos y carga sin comunicar éxitos no confirmados. El tratamiento de información de usuarios y vehículos exige acceso mínimo, retención definida y respaldos controlados. Deben verificarse contraste, navegación, mensajes de error y compatibilidad Android con dispositivos reales.

11. Infraestructura y ambientes

Tabla 5. Ambientes de ejecución y evidencia requerida

Ambiente	Objetivo	Condición de aceptación
Desarrollo	Construcción y prueba local	Configuración segregada y datos sintéticos
QA / sandbox	Validar integración API, Wompi y proveedores	Evidencias fechadas, versión y escenarios de éxito/error
Producción	Disponibilidad para usuarios	Confirmar despliegue, TLS, migraciones, respaldo, monitoreo y recuperación

Los repositorios contienen configuración de Docker, Compose, Expo/EAS y frontend Vite, y la documentación describe Render como plataforma de alojamiento. La presencia de esos archivos no acredita qué versión se encuentra actualmente desplegada. Debe correlacionarse cada despliegue con su commit, variables de entorno por nombre, registro de migraciones y pruebas de salud.

12. Plan de trabajo y entregables

Tabla 6. Fases de consolidación y validación

Fase	Entregable / comprobación
1. Consolidación documental	Requisitos, arquitectura, matriz de tecnologías y contratos
2. Integración de datos	Migraciones Liquibase aplicadas y restricciones verificadas
3. Integraciones externas	Escenarios Wompi sandbox, SMTP y push con evidencias
4. Calidad del producto	E2E, permisos, compatibilidad y pruebas de rendimiento
5. Entrega	Manual, referencias de versión, monitoreo, respaldos y recuperación

Estas fases son una ruta de validación propuesta y no un cronograma comprometido. Fechas, dedicación y porcentaje de ejecución deben fijarse mediante planificación aprobada.

13. Equipo y responsabilidades

Tabla 7. Participación del equipo del proyecto

Integrante	Participación descrita
José Stiven Perdomo Martínez	Desarrollo full stack, integración y documentación, según distribución del equipo
Santiago Gordo Pérez	Liderazgo de proyecto, desarrollo full stack, integración, pruebas y seguimiento

La distribución definitiva de tareas puede detallarse en los registros del proyecto. Un rol de pruebas mencionado en estimaciones presupuestales no debe confundirse con una persona contratada o integrante efectivo.

14. Estimación económica histórica

El documento de propuesta previo consigna un presupuesto referencial total de $94.757.900 COP. Se mantiene únicamente como antecedente de planificación y no como costo ejecutado, oferta vigente ni presupuesto recalculado con tarifas actuales.

Tabla 8. Resumen del presupuesto histórico

Rubro	Valor histórico (COP)
Personal	$90.000.000
Hardware	$3.042.900
Infraestructura	$1.715.000
Software	$0
Total	$94.757.900

El esquema histórico de pagos plantea 20 % al inicio, 30 % al diseño, 30 % al desarrollo y pruebas y 20 % a la entrega, sin evidencia de pagos realizados. Para una cotización real sería necesario actualizar horas de dedicación, infraestructura, tarifas de terceros, soporte, impuestos y recurrencia.

15. Pruebas y criterios de validación

Se identifican archivos de pruebas para backend, panel y aplicación móvil, así como escenarios de rendimiento documentados previamente. No se ejecutó un nuevo conjunto de pruebas para establecer un conteo de casos aprobados sobre los commits del 4 de noviembre de 2026. Los resultados históricos no deben presentarse como resultados actuales.

Tabla 9. Evidencia técnica y estado de verificación

Área	Identificado	Verificación que falta
Backend	Pruebas y servicios para reservas, permisos y pagos	Ejecución reproducible por commit y pruebas integradas
Móvil y panel	Flujos y pruebas de componentes	Build, compatibilidad, integración y E2E
PostgreSQL	18 tablas y 24 changeSets declarados	Estado real de Liquibase y esquema desplegado
Wompi / proveedores	Flujos de validación y entrega en código	Transacciones sandbox, webhooks y entrega real
Rendimiento	Escenarios k6 documentados previamente	Reproducir con la versión, infraestructura y carga objetivo

La aceptación debe cubrir horarios y solapamiento de reservas, accesos por rol, estados operativos, firma y verificación Wompi, casos rechazados y duplicados, recuperación segura, notificaciones y comportamiento ante pérdida de conectividad.

16. Mantenimiento, riesgos y sostenibilidad

Tabla 10. Principales riesgos y medidas de mitigación

Riesgo	Medida de mitigación
Diferencia entre esquema declarado y desplegado	Comprobar DATABASECHANGELOG y validación JPA
Falla o discrepancia en verificación de pago	Pruebas Wompi sandbox, idempotencia y conciliación
Acceso indebido a recursos	Pruebas por rol, titularidad y expiración de JWT
Dependencia de proveedores externos	Control de timeout, reintentos y manejo de errores
Escalamiento de rate limit en memoria	Evaluar mecanismo compartido en despliegue multinodo
Obsolescencia de dependencias	Versionado y pruebas de regresión controladas

El mantenimiento debe contemplar correcciones funcionales, actualización de dependencias, respaldos, revisión de registros, adaptación a cambios de plataformas y mejoras aprobadas dentro del alcance. Funcionalidades adicionales requieren gestión formal de cambios.

17. Conclusiones

La propuesta técnica se apoya en una implementación cliente-servidor con backend monolítico modular y separación entre interfaz, lógica de negocio y base de datos. Las elecciones de React Native/Expo, React/Vite, Spring Boot, PostgreSQL/Liquibase y Wompi responden a los requisitos de movilidad, administración web, control transaccional e integración de pagos.

El principal criterio para el cierre de entrega es disponer de evidencias integradas, trazables a la versión publicada y sin información sensible. La documentación de los componentes y la existencia del código no

sustituyen la verificación operativa del esquema, de los proveedores externos ni de las pruebas de extremo a extremo.

18. Referencias

Equipo del proyecto Lava Rápido Vehicular. (2026). Especificación de requisitos de software (SRS) [Documento interno]. Servicio Nacional de Aprendizaje.

Equipo del proyecto Lava Rápido Vehicular. (2026). Documentación de análisis del software: Vistas dinámicas [Documento interno]. Servicio Nacional de Aprendizaje.

Expo. (s. f.). Expo documentation. https://docs.expo.dev/

Liquibase. (s. f.). Liquibase documentation. https://docs.liquibase.com/

Meta Open Source. (s. f.). React. https://react.dev/

PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation. https://www.postgresql.org/docs/

Spring. (s. f.). Spring Boot documentation. https://docs.spring.io/spring-boot/

Wompi. (s. f.). Documentación para desarrolladores. https://docs.wompi.co/

Vite. (s. f.). Vite guide. https://vite.dev/guide/